

7.1.

```
PS C:\Users\1> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
client-114064548    latest             181618aa3df2       4 minutes ago      1.8GB
server-114064548    latest             c94862351efa       5 minutes ago      1.8GB
PS C:\Users\1> |
```

```
PS C:\Users\1> docker run -it --rm --network host --name server server-114064548
Server started listening on port 8888
Received message from Client: Hello from 114064548
Response sent to Client
```

```
PS C:\Users\1> docker run -it --rm --network host --name client client-114064548
Hello message sent.
Hello from Server
```

7.2.

```
PS C:\Users\1> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ea058ae83992        box-bridge          bridge              local
```

```
PS C:\Users\1> docker network inspect box-bridge
[
  {
    "Name": "box-bridge",
    "Id": "ea058ae839920c483d6af9f65b42b45a7ed43fe7811bf4987426c5148fcb5363",
    "Created": "2025-10-19T06:11:17.703172577Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "02ab2c8ca466d16a58392a7ad4aac9ef465b4c2f1bbd0eb4262bf3527ff64cc5": {
        "Name": "box2",
        "EndpointID": "fed18aa06c32d4b23366803da0b0242a1fee019f5d5ecc034a4d3132002c6e0a",
        "MacAddress": "66:44:b1:73:5c:cc",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "0ff6b76eec85df12b973b1d467a3544eb955e0b6f73b539920298f578c71b99e": {
        "Name": "box1",
        "EndpointID": "ee88fe2197b8441d4558b2c35ff891b482c8d8678f4dd602454bae5b0486b9b5",
        "MacAddress": "42:1a:f1:52:46:7e",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {}
  }
]
```

```
PS C:\Users\1> docker exec -it box1 sh
/ # ping box2
PING box2 (172.18.0.3): 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=64 time=0.112 ms
64 bytes from 172.18.0.3: seq=1 ttl=64 time=0.102 ms
64 bytes from 172.18.0.3: seq=2 ttl=64 time=0.084 ms
64 bytes from 172.18.0.3: seq=3 ttl=64 time=0.112 ms
|
```

7.3.

1. You need to use docker volume create to create a volume and share the volume between two containers.

```
PS C:\Users\1> docker volume create 114064548
114064548
```

建立 Volume：輸入以上指令來建立一個 Volume

```
PS C:\Users\1> docker run --rm -v 114064548:/data alpine sh -c "echo 'Hello from Container 1' > /data/test.txt"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
2d35ebdb57d9: Pull complete
Digest: sha256:4b7ce07002c69e8f3d704a9c5d6fd3053be500b7f1c69fc0d80990c2ad8dd412
Status: Downloaded newer image for alpine:latest
```

啟動一個 alpine container，將 114064548 掛載到 container 內部的 /data 資料夾，然後寫入一個檔案。

--rm: container 停止後自動刪除。

-v my-shared-data:/data: 把 my-shared-data 這個 volume 掛載到 container 裡的 /data。

sh -c "...": 在 container 內執行 "..." 這串指令。

```
PS C:\Users\1> docker run --rm -v 114064548:/data alpine cat /data/test.txt
Hello from Container 1
```

啟動第二個 Container (讀取者)：啟動另一個 container，也把同一個 114064548 掛載到它的 /data 資料夾，然後讀取剛剛的檔案。會看到 Hello from Container 1，這證明了第二個 container 成功讀到了第一個 container 寫入的資料。

2. You need to use bind mounts to share a directory between two containers.

```
PS C:\Users\1> mkdir my-host-share

目錄：C:\Users\1

Mode                LastWriteTime         Length Name
----                -
d-----          2025/10/19 下午 02:41             my-host-share
```

建立一個叫 my-host-share 的資料夾。

```
PS C:\Users\1> docker run --rm -v $HOME\my-host-share:/shared alpine sh -c "echo 'Hello from Host' > /shared/hostfile.txt"
PS C:\Users\1> docker run --rm -v $HOME\my-host-share:/114064548 alpine cat /114064548/hostfile.txt
Hello from Host
```

啟動第一個 **Container (寫入者)**： 啟動一個 container，這次 -v 的參數是主機的絕對路徑，並啟動第二個 **Container (讀取者)**： 我們啟動第二個 container，掛載同一個主機資料夾，並讀取那個檔案。這證明了第二個 container 成功讀到了第一個 container 寫入的資料。

```
PS C:\Users\1> Get-Content $HOME\my-host-share\hostfile.txt
Hello from Host
```

證明 container 裡的檔案真的被寫入到你的 Windows 電腦上了。

docker volume create 會建立一個由 Docker 引擎統一管理的儲存空間 (Volume)，它獨立於 Container 的生命週期，適合用來儲存資料庫等持久性資料。

bind mounts 則是將主機 (Host) 電腦上的一個特定路徑 (如 C:\my-code)，直接映射到 Container 內部。它主要用於在主機和 Container 之間共享檔案，例如將主機的程式碼掛載到 Container 中執行。

兩者的主要差異在於：Volume 由 Docker 管理，Bind Mount 由使用者直接指定主機路徑。

3. Answer the following questions: (20 points)

- Please explain the difference between Docker Container and Virtual Machine. *(10 points)

VM (虛擬機) 虛擬化的是**硬體**。它會模擬出一整套 CPU、RAM 和硬碟，然後在上面安裝一個完整的「訪客作業系統」(Guest OS)。

Container (容器) 虛擬化的是**作業系統**。它會共享主機 (Host) 的作業系統核心 (Kernel)，只打包應用程式必要的函式庫。這使得 Container 非常輕量 (MB 等級)、啟動極快 (秒級)，但隔離性較弱；而 VM 非常笨重 (GB 等級)、啟動很慢 (分鐘級)，但隔離性極強。

- Docker is more unsafe than virtual machine, please explain why and what's causing this issue. (10 points)

在 Docker 架構中，所有 Container 都共用同一個主機的作業系統核心。如果攻擊者在某個 Container 中找到並利用了**核心的漏洞**，他就有可能「逃脫」(escape) Container 的限制，進而取得整個主機的最高權限，並控制主機上所有其他的 Container。VM 則沒有這個問題。每台 VM 都有自己獨立的 Guest OS 核心，並受到 Hypervisor 的硬體層級隔離。攻破一台 VM 並不會直接威脅到主機或其他 VM。

8.1

```
PS C:\Users\1\Downloads\hw2\hw2\k8s> kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
socket-server-5f98dbf6ff-wk95z     0/1     ErrImageNeverPull    0           32s
PS C:\Users\1\Downloads\hw2\hw2\k8s> kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
socket-server      0/1     1            0           37s
PS C:\Users\1\Downloads\hw2\hw2\k8s> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1        <none>         443/TCP           10m
socket-server-service NodePort    10.99.161.107    <none>         8888:30080/TCP    34s
```

What is Kubernetes? Why do we need it? (5 points)

Kubernetes (K8s) 是一個開源的容器調度平台 (**Container Orchestration Platform**)。

我們需要它，因為在現代應用中，我們可能需要運行成百上千個 **container**。如果沒有 **K8s**，手動管理這些 **container** 的部署、擴展、網路和健康狀態會是一場災難。**K8s** 幫我們自動化了這一切，它提供了自我修復（自動重啟失敗的 **container**）、彈性擴展（自動增減 **container** 數量）以及服務發現和滾動更新等關鍵功能。

Please explain the concepts of Deployment, Service and Pod. (5 points)

- **Pod (豆莢):** 這是 **Kubernetes** 中最小的部署單位。它像是一個「包裝盒」，裡面可以包含一個或多個緊密關聯的 **container**，並共享網路和儲存資源。
- **Service (服務):** 這是一個穩定的網路端點，就像一個「服務台」。**Pod** 的 IP 會變動，但 **Service** 的 IP 固定不變。它會接收請求，並自動將流量負載平衡到它後面所對應的 **Pod** 上。
- **Deployment (部署):** 這是你用來描述「期望狀態」的藍圖。你告訴 **Deployment** 你要"3 個 **Pod**"，它就會去建立並管理 **ReplicaSet** 來確

保「實際狀態」永遠符合你的「期望狀態」，它也是更新應用程式版本的主要物件。

Why k8s use pod to manipulate the application instead of the container. (5G points)

因為 Pod 提供了更高層級的抽象，允許「多個 container 被綁定為一個單元」。

在實際應用中，很多程式不只是一個 container，它們常需要一個主程式 container 搭配多個「輔助 container」(Sidecar)，例如日誌收集器或網路代理。這組 container 需要緊密地一起工作、共享網路 (如 localhost) 和儲存。

如果 K8s 直接操作 container，就很難定義「這幾個 container 必須總是在同一台機器上、一起啟動、一起停止」。透過 Pod 這個「原子單元」，K8s 確保了這個最小部署單位內的 container 永遠「同生共死」，完美地解決了這個問題。

What is Replica Set in Kubernetes? (5 points)

ReplicaSet (副本集) 就像一個「團隊經理」，它的唯一工作就是確保「指定數量的 Pod 副本 (replicas) 永遠在運行」。

它會持續監控 Pod。如果一個 Pod 意外死掉或被刪除，ReplicaSet 會立刻啟動一個新的 Pod 來替補，以維持你期望的數量 (例如 3 個)。在實務上，你幾乎不會自己手動建立 ReplicaSet，而是透過 Deployment 來自動建立與管理它。

Why can the client successfully communicate through port 30080 when the server is listening on port 8888? (10 points)

這是因為你使用了一種稱為 NodePort 的 Service (服務) 類型，Service 在這裏扮演了「網路總機」的角色，它負責轉接和翻譯通訊埠。

這裏有兩個不同的通訊埠：NodePort (外部埠 30080) 和 targetPort (Pod 內部埠 8888)。

連線流程如下：

1. **Client** 將請求發送到 **[Node 節點的 IP]:30080**。
2. **Kubernetes** 的 **Service** 在 **30080** 埠收到了這個流量。
3. **Service** 查看設定，發現這個流量的目標 (**targetPort**) 是 **8888**。
4. **Service** 接著將這個流量轉發 (**forward**) 到它所管理的某一個 **Pod** 的 **IP** 位址，並將目的埠從 **30080** 轉譯 (**translate**) 成 **8888**。
5. **Server Pod** 最終在它熟悉的 **8888** 埠上收到了請求，並成功回應。